TOM Genesis

Seeded Compilation Profile 1.0 over TOMAGI ABI 1.0

Tom Klootwijk - requester-supplied attribution

1 September 2026

Contents

1 Status, purpose, and evidence boundary

This document is the normative definition of **TOM Seeded Compilation Profile 1.0 over TOMAGI ABI 1.0**. It closes the executable chain between the canonical TOM seed and the fixed-width TOMAGI machine:

```
canonical seed bytes
-> exact parse and token resolution
-> typed, content-addressed executable definitions
-> deterministic definition evaluation
-> State64 and Cell48 construction
-> canonical Cell48 lowering
-> unchanged TOMAGI 1.0 .tmg serialization
-> ordered TOMAGI execution and EMIT records
-> domain-neutral byte materialization
-> byte-identical artifact and replay certificate
```

The profile is an additive source-compilation and materialization contract. It does not alter the 128-byte `.tmg` header, the 64-byte `State64` record, the 48-byte `Cell48` record, the sixteen opcode numbers, or the state-transition equations of TOMAGI 1.0.

Requester-supplied attribution and identifiers are retained from the project corpus and are not independently verified. This specification does not establish patentability, worldwide novelty, universal performance, learned generalization, or a physical-device result.

1.1 Conformance language

The words **MUST**, **MUST NOT**, **SHOULD**, and **MAY** are normative. A seeded-conformance implementation MUST satisfy every requirement marked MUST. An implementation may additionally support legacy TOMAGI source documents, provided legacy behavior remains byte-compatible.

1.2 Delivered implementation status

The shipped implementation demonstrates:

- exact parsing of the 244-byte root seed;
- content-addressed token and definition registries;
- a 38-operation generic formal algebra;
- explicit `State64` and `Cell48` constructors;
- deterministic dependency evaluation and key-sorted lowering;
- unchanged TOMAGI ABI 1.0 binary output;
- generic one-to-four-byte `EMIT` materialization;
- migrated `polar_loop` and `exact19_rule` programs without top-level handwritten `cells` arrays;
- the 32 x 61 TOM Seed Raster;
- a formal-operation-algebra proof artifact;
- full Python/C state, trace, lineage, `EMIT`, and byte equality;
- deterministic rejection and clean source-capsule replay;
- a validation certificate whose own bytes are emitted by TOMAGI.

No new GPU-device dispatch evidence is claimed by this release.

2 Profiles and compatibility

2.1 TOMAGI ABI 1.0

TOMAGI ABI 1.0 remains the binary execution authority. The version word is `0x00010000`; the header is 128 bytes; each state is 64 bytes; and each cell is 48 bytes. The hot evaluator remains integer-only and uses the existing sixteen opcodes.

2.2 Legacy cell mode

A JSON source document without `compilation_profile = "TOM-Seeded-Compilation-Profile-1.0"` is compiled by the legacy compiler. Its top-level `cells` array remains authoritative. Existing legacy examples are retained under `examples/legacy/` and rebuild byte-for-byte to their archived `.tmg` files.

2.3 Seeded conformance mode

A source document declaring:

```
{
  "tomagi_version": "1.0.0",
  "compilation_profile": "TOM-Seeded-Compilation-Profile-1.0"
}
```

enters seeded conformance mode. In this mode:

1. the exact canonical seed and token registry MUST resolve;

2. every definition MUST be hash-valid and executable;
3. every definition MUST be reachable from the selected root;
4. top-level handwritten `cells` MUST be rejected;
5. the root MUST evaluate to a TOMAGI program;
6. every lowered cell MUST be attributable to an executable definition;
7. the resulting binary MUST use unchanged TOMAGI ABI 1.0.

3 Canonical root genome

3.1 Exact bytes

The authoritative root file is:

`TOM_seed_genome_2026-09-01.txt`

Its exact one-line ASCII content is:

```
TOM1[TopologicalOpenModular]|TomKlootwijk|1990-07-10|NL200678942|2026-09-01|LUTlogp^{Klein,SDF0@Def}(rho,theta,t->;phi
    ,dt,d2,J,v,a,j1)>P1>L2_BST^b>ASweepCone(T,apex)>Pi[pyrSide,circle,sphere]>support>compatibility>guard>event>
    transition>lineage
```

The canonical boundary values are:

Property	Normative value
Encoding	ASCII subset of UTF-8
Byte length	244
Terminal line ending	Forbidden
SHA-256	`d1417a3136772c0cf3eddcd4962ce07d42cbf87616f7b5bae09fc652d9b807b5`

No whitespace trimming, Unicode normalization, case folding, punctuation repair, or permissive tokenization is allowed.

3.2 Exact profile-1.0 grammar

```
seed       = header, "|", pipeline ;
header     = "TOM1[TopologicalOpenModular]", "|",
             "TomKlootwijk", "|",
             "1990-07-10", "|",
             "NL200678942", "|",
             "2026-09-01" ;
pipeline   = lut, ">P1>L2_BST^b>", cone, ">", projection,
             ">support>compatibility>guard>event>transition>lineage" ;
lut        = "LUTlogp^{Klein,SDF0@Def}(rho,theta,t->;",
             "phi,dt,d2,J,v,a,j1)" ;
cone       = "ASweepCone(T,apex)" ;
projection = "Pi[pyrSide,circle,sphere]" ;
```

The parser MUST first verify length and digest, then decode ASCII, then match this exact grammar. A byte mutation, a terminal line feed, or any punctuation change invalidates the seed.

3.3 Token registry

The versioned registry is `spec/seeded/token_registry_1_0.json`. It is bound to the seed digest and has content hash:

`sha256:b14140cf9800e186701557ed982d692931966ea957e5790c1e6b4989e854c609`

It resolves the following ordered 28-token sequence:

```
LUTlogp, Klein, SDF0@Def, rho, theta, t->, phi, dt, d2, J, v, a,
j1, P1, L2_BST^b, ASweepCone, T, apex, Pi, pyrSide, circle, sphere,
support, compatibility, guard, event, transition, lineage
```

A conforming registry MUST contain one unique entry for every token and MUST declare this ten-phase order:

Phase	Name	Purpose
0	parse	Verify and parse literal sources.
1	normalize	Establish exact typed values and coordinate forms.
2	resolve	Resolve definitions and the `SDFO@Def` interface.
3	construct	Construct literals, records, state, relations, and graph components.
4	transform	Apply bounded deterministic transforms.
5	support	Admit declared locally relevant candidates.
6	compatibility	Apply declared type, route, phase, sheet, orientation, or address compatibility.
7	guard/root	Evaluate declared relation or predicate guards and form event selectors.
8	transition	Construct cells, byte emission, and atomic route effects.
9	lineage/log	Build the final program and preserve replay provenance.

4 Canonical JSON and content addressing

4.1 Canonical JSON bytes

For a JSON value `x`, `canonicalJSON(x)` is UTF-8 encoding of JSON with:

- object keys sorted lexicographically;
- compact separators `,` and `:`;
- no insignificant whitespace;
- Unicode characters encoded directly rather than ASCII-escaped;
- JSON-domain values only.

The reference implementation is equivalent to:

```
json.dumps(
    x,
    sort_keys=True,
    ensure_ascii=False,
    separators=(",", ":"),
).encode("utf-8")
```

4.2 Definition content hash

For definition `d`:

```
content_hash(d) = "sha256:" || hex(
    SHA256(canonicalJSON(d with the content_hash member removed))
)
```

The compiler MUST recompute and compare this value before evaluating the definition. A mismatch is a compilation error.

4.3 Token-registry content hash

The token registry uses the same rule. It MUST also declare the canonical seed SHA-256 and the exact 28-token sequence.

5 Seeded source document

The machine-readable schema is `spec/tom_seeded_program.schema.json`, Draft 2020-12.

A seeded document contains:

Field	Requirement
`tomagi_version`	MUST equal `1.0.0`.
`compilation_profile`	MUST equal `TOM-Seeded-Compilation-Profile-1.0`.
`seed_genome`	Relative file path, byte count 244, and canonical digest.
`token_registry`	Relative file path and registry content hash.
`root_definition`	ID of the final program-producing definition.
`limits`	Explicit global resource budgets.
`definitions`	Non-empty array of content-addressed executable definitions.
`cells`	MUST NOT appear in seeded conformance mode.

Relative paths are resolved from the source document directory. Absolute seed and registry paths are rejected.

6 Definition record and graph

6.1 Required definition fields

Every definition is a first-class executable record:

```
d = (
  id, kind, domain, codomain,
  phase, ordinal,
  dependencies, inputs,
  operation, parameters, limits,
  provenance, zero_relation,
  content_hash
)
```

Field	Normative meaning
`id`	Unique non-empty definition identifier.
`kind`	Human-readable typed role; included in the hash.
`domain`	Declared input/domain description.
`codomain`	Runtime evaluator type name.
`phase`	Integer `0..9`.
`ordinal`	Non-negative order within a phase.
`dependencies`	Definition IDs that MUST resolve and precede this node.
`inputs`	Ordered value inputs; MUST be a duplicate-free subset of dependencies.
`operation`	One operation from the profile algebra.
`parameters`	Literal operation parameters.
`limits`	Hashed per-definition constraints that are enforced after evaluation.
`provenance`	Literal source role and crosswalk metadata.
`zero_relation`	Typed `SDF0@Def` interface declaration.

Field	Normative meaning
`content_hash`	Canonical SHA-256 content address.

6.2 Dependency and schedule rules

The compiler MUST:

1. reject duplicate definition IDs;
2. reject duplicate dependencies or duplicate inputs;
3. reject unresolved dependency IDs;
4. require `inputs` to be a subset of `dependencies`;
5. reject dependency cycles;
6. reject duplicate `(phase, ordinal)` slots;
7. require every dependency slot to precede the dependent slot;
8. compute a stable topological order using `(phase, ordinal, id)` ordering for ready nodes;
9. require every definition to be reachable from `root_definition`;
10. require the root to be the final scheduled definition.

A definition is therefore not decorative metadata: every reachable definition is hash-verified, evaluated, type-checked, limit-checked, and recorded in the compile manifest.

7 Type system

The profile evaluator has the following finite types:

Type	Representation
`bytes`	Immutable byte sequence.
`string`	Unicode scalar sequence, encoded only by an explicit operation.
`array`	Canonical JSON array.
`record`	Canonical JSON object.
`json`	Any canonical JSON value.
`int`	Mathematical integer; fixed-width operations explicitly normalize it.
`bit`	Integer `0` or `1`.
`state`	One `State64` value.
`cell`	One unresolved `CellPlan`.
`cells`	Finite ordered list of unresolved `CellPlan` values.
`program`	A complete TOMAGI ABI 1.0 program bundle.

Operation output type MUST equal the definition's declared `codomain` exactly.

8 Formal operation algebra

Profile 1.0 defines 38 domain-neutral operations. Artifact formats and domain behavior MUST be expressed by composition of these operations and TOMAGI cells, not by host-side special cases.

8.1 Seed and canonical literals

Operation	Inputs	Output	Exact behavior
`seed.bytes`	0	`bytes`	Return the already verified canonical seed bytes.
`literal.utf8`	0	`bytes`	UTF-8 encode `parameters.text`.
`literal.hex`	0	`bytes`	Decode `parameters.hex` as hexadecimal octets.
`literal.json`	0	`json`	Canonical-JSON round trip of `parameters.value`.
`literal.int`	0	`int`	Parse the declared integer, including prefixed string forms.
`literal.string`	0	`string`	Return the declared string.
`literal.array`	0	`array`	Canonical-JSON clone of the declared array.
`literal.record`	0	`record`	Canonical-JSON clone of the declared object.
`string.utf8`	1 `string`	`bytes`	Encode the input string as UTF-8.

8.2 Bounded sequence operations

Operation	Inputs	Output	Exact behavior
`bytes.concat`	1+ `bytes`	`bytes`	Compatibility alias for ordered byte concatenation.
`bytes.slice`	1 `bytes`	`bytes`	Non-negative half-open slice `[start:stop]`.
`bytes.repeat`	1 `bytes`	`bytes`	Repeat `count` times within `max_repeat`.
`sequence.concat`	1+ same-type sequences	same type	Concatenate `bytes`, `string`, or `array` inputs in declared input order.
`sequence.slice`	1 sequence	same type	Non-negative half-open slice.
`sequence.repeat`	1 sequence	same type	Repeat within global repeat and sequence budgets.
`sequence.indexed_map`	1 `array`	`array`	Apply one declared finite map: `identity`, `index`, `pair`, `u32.add_index`, or `u32.xor_index`.

Negative slice bounds are rejected. No implicit iteration over an unbounded source is permitted.

8.3 Fixed-width integer and bit operations

All `u32` operations act in `Z/(2^32 Z)`. Shift and rotate counts are masked with `31`.

Operation	Inputs	Output	Equation
`u32.normalize`	1 integer	`int`	`x & 0xffffffff`
`i32.normalize`	1 integer	`int`	Two's-complement signed interpretation of `u32(x)`.
`u32.add`	2 integers	`int`	`u32(a + b)`
`u32.xor`	2 integers	`int`	`u32(a) XOR u32(b)`
`u32.and`	2 integers	`int`	`u32(a) AND u32(b)`
`u32.or`	2 integers	`int`	`u32(a) OR u32(b)`
`u32.shl`	2 integers	`int`	`u32(u32(a) << (b & 31))`
`u32.shr`	2 integers	`int`	Logical `u32(a) >> (b & 31)`
`u32.rotl`	2 integers	`int`	32-bit rotate-left.
`u32.popcount`	1 integer	`int`	Population count of `u32(x)`.
`bit.parity`	1 integer	`bit`	`popcount32(x) mod 2`.

8.4 Predicates and binary selection

Operation	Inputs	Output	Exact behavior
`predicate.eq`	2 equal-typed values	`bit`	Return `1` exactly when the values are equal.
`predicate.le`	2 integers	`bit`	Return `1` exactly when `a <= b`.
`predicate.zero`	1 integer/bit	`bit`	Return `1` exactly when the input is zero.
`select.binary`	selector, false value, true value	route type	Selector MUST be `0` or `1`; route types MUST match; input order is false then true.

These predicates are declared program semantics. They are not confidence thresholds, safety margins, or hidden recovery policies.

8.5 Canonical encoding

Operation	Inputs	Output	Exact behavior
`json.canonical_bytes`	JSON-domain value	`bytes`	Encode canonical JSON; append one LF only when `terminal_newline` is true.

8.6 State, cell, graph, emission, and program construction

Operation	Inputs	Output	Exact behavior
`tomagi.state64`	0	`state`	Construct all sixteen state fields with exact `i32`/`u32` range checks. Omitted fields are zero.

Operation	Inputs	Output	Exact behavior
`tomagi.cell`	0	`cell`	Construct one unresolved Cell48 plan from key, opcode, flags, four args, two successor IDs, payload, and aux.
`tomagi.cells.concat`	1+ cell/cells	`cells`	Flatten cell inputs in declared order.
`tomagi.graph.sequence`	1+ cell/cells	`cells`	Normative graph-construction alias with the same finite flattening semantics.
`tomagi.emit_bytes`	1 `bytes`	`cells`	Partition bytes into 1-4 byte chunks and construct an ordered EMIT chain.
`tomagi.program`	seed, optional state, cells	`program`	Verify seed binding; sort by canonical key; resolve successor IDs; construct the unchanged ABI program.

9 Limits and bounded evaluation

9.1 Global default limits

The reference implementation applies these defaults when a document does not lower them:

Limit	Default
`max_definitions`	4,096
`max_dependencies_per_definition`	256
`max_literal_bytes`	16 MiB
`max_output_bytes`	16 MiB
`max_cells`	1,048,576
`max_operation_steps`	2,000,000
`max_ticks`	2,000,000
`max_repeat`	1,000,000
`max_sequence_items`	1,000,000
`max_recursion_depth`	0

Documents MAY declare lower non-negative values. Profile 1.0 has no recursive definition operation; any non-zero recursion-depth declaration is rejected.

9.2 Per-definition limits

A definition's `limits` object is content-addressed and enforced. Recognized constraints include exact/minimum/-maximum byte counts, character counts, array items, record fields, canonical JSON byte counts, integer ranges, bit values, exactly sixteen state words, cell counts, and one-to-four bytes per generated EMIT cell. Unknown limit keys are rejected so that a purported limit cannot be silently ignored.

10 `SDF0` and `SDF0@Def`

The two names are related but MUST remain distinct.

10.1 Hot opcode SDF0

SDF0 is unchanged TOMAGI opcode 6. For a key in the finite compiled LUT domain, it sets the current residual to literal zero, marks zero status, and produces its declared branch. Outside the finite cell domain, the address is undefined. It does not contract a state toward a surface and it is not a universal constant field over all 64-bit words.

10.2 Definition interface SDF0@Def

Every seeded definition MUST contain:

```
{
  "zero_relation": {
    "interface": "SDF0@Def",
    "domain": "...",
    "codomain": "...",
    "semantics": "...",
    "lowering": "..."
  }
}
```

This declares the typed meaning of zero for that definition. Examples include zero hash discrepancy, zero byte discrepancy, zero graph discrepancy, a cone residual, a sphere residual, or successful emission completeness.

The lowering member is checked specially for relation opcodes:

- a tomagi.cell definition with opcode SDF0 MUST declare lowering SDF0;
- opcode CONE MUST declare lowering CONE;
- opcode SPHERE MUST declare lowering SPHERE.

A cone definition may therefore expose SDF0@Def while lowering the runtime relation test to CONE. The meta-interface does not rename or replace the hot opcode.

11 Definition evaluation

For source document D, exact seed g, registry R, and budgets B, compilation performs:

```
verify_seed(g)
parse_exact(g)
verify_registry(R, g)
verify_schema(D)
verify_definition_hashes(D.definitions)
order = stable_topological_order(D.definitions)
assert all definitions reachable from D.root_definition
for id in order:
    values[id] = evaluate(operation[id], ordered inputs[id], parameters[id])
    assert type(values[id]) == codomain[id]
    enforce limits[id]
assert values[root] is program
serialize TOMAGI ABI 1.0
write definition/output/lowering manifest
```

Evaluation order is observable in the compile manifest. The compiler records every definition content hash, operation, phase, ordinal, dependencies, inputs, zero relation, output type, output hash, and relevant size.

12 Cell48 lowering

12.1 Unresolved CellPlan

Before sorting, a cell plan contains:

```
cell_id, key_hi, key_lo, opcode, flags,
arg0, arg1, arg2, arg3,
next_id_0, next_id_1,
payload, aux, definition_id,
optional source byte offset/count
```

12.2 Canonical key input

A cell key is either:

- one exact unsigned 64-bit integer; or
- an object containing `rho`, `theta`, `tick`, and `phi`, packed with the existing 20/18/14/12 contiguous key codec.

12.3 Sorting and successor resolution

The compiler MUST:

1. reject duplicate cell IDs;
2. reject duplicate canonical 64-bit keys;
3. sort cell plans by unsigned 64-bit key;
4. build the post-sort `cell_id -> index` map;
5. reject unresolved successors;
6. rewrite both successor IDs to post-sort indices;
7. reject a missing entry cell;
8. construct immutable `Cell48` records.

The compile manifest records a definition-to-cell crosswalk for every cell, including source offsets and byte chunks for generated EMIT cells.

12.4 Initial state

The recommended seeded form constructs the initial state through a `tomagi.state64` definition and names it with `parameters.state_input`. The implementation retains an `initial_state` parameter fallback for source compatibility, but a document MUST NOT supply both forms.

12.5 Seed binding in the program header

To ensure that the emitted `.tmg` bytes are causally bound to the canonical genome without changing transition semantics:

```
genome_tag = little_endian_u32(first four bytes of SHA256(seed_bytes))
program.flags = u32(declared_program_flags XOR genome_tag)
```

For the canonical seed, `genome_tag` is `830095825` (`0x317a41d1`). The header flag word is not interpreted by the hot TOMAGI 1.0 transition algebra.

13 Unchanged TOMAGI binary ABI

13.1 Header

The `.tmg` byte layout remains:

Byte	Size	Field
0	8	ASCII `TOMAGI1\0`
8	4	Version `0x00010000`
12	4	Program flags
16	4	Cell count
20	4	Entry index
24	4	Runtime seed
28	4	Default tick count
32	4	Cell size, 48
36	4	State size, 64
40	24	Reserved zero words
64	64	Initial `State64`
128	`48N`	Key-sorted `Cell48[N]`

Total length remains `128 + 48N` bytes.

13.2 State64

The state remains sixteen 32-bit words: four coordinates, four first differences, orientation, sheet, branch, cell, lineage, output, residual, and status.

13.3 Cell48

The cell remains twelve 32-bit words: key high/low, opcode, flags, four signed args, two successors, payload, and aux.

13.4 Opcode semantics

All sixteen opcode numbers and transition equations remain those of TOMAGI 1.0. The seeded compiler only constructs records and the materializer only observes executed `EMIT` records.

14 Generic byte-emission profile

14.1 Flag allocation

For opcode `EMIT` only:

Flag bits	Meaning
bit 0	Existing TOMAGI `EMIT` halt bit.
bit 8	`1` means seeded byte-emission mode; `0` means legacy symbolic token mode.
bits 10:9	`byte_count - 1`, encoding counts 1 through 4.
other bits	Retain their existing/reserved meaning.

Constants are:

```
FLAG_EMIT_BYTES      = 1 << 8
FLAG_EMIT_COUNT_MASK = 3 << 9
byte_count           = ((flags & FLAG_EMIT_COUNT_MASK) >> 9) + 1
```

14.2 Payload packing

For a chunk `b[0:n]`, where `1 <= n <= 4`:

```
payload = b0 | (b1 << 8) | (b2 << 16) | (b3 << 24)
```

Missing high bytes are zero. Materialization returns the lowest `byte_count` bytes in little-endian order.

14.3 Compiler-generated chain

`tomagi.emit_bytes`:

1. rejects an empty input;
2. rejects input larger than `max_output_bytes`;
3. partitions bytes into consecutive chunks of at most four;
4. assigns consecutive 64-bit keys from `key_start`;
5. generates stable IDs `<prefix>:000000`, `<prefix>:000001`, and so on;
6. stores the source byte offset in `aux` and the compile manifest;
7. makes both branches advance to the next generated cell;
8. makes the final cell self-loop;
9. applies the existing halt bit to the final cell when requested.

14.4 Domain-neutral materialization algorithm

```
output = empty byte sequence
for record in execution_trace order:
    cell = program.cells[record.cell_before]
    if cell.opcode != EMIT:
        continue
    if byte-mode bit is zero:
        reject in strict byte mode, or record symbolic emission in mixed mode
        continue
    count = decode bits 10:9 plus one
    output += little_endian(cell.payload)[0:count]
return output
```

The materializer has no PBM, JSON, text, image, mesh, audio, or report-specific branch.

15 Genesis proof artifacts

15.1 TOM Seed Raster

The exact root seed is 244 bytes = 1,952 bits. Since `32 x 61 = 1,952`, the canonical artifact is:

```
header = ASCII bytes "P4\n32 61\n"   (9 bytes)
raster = exact canonical seed bytes    (244 bytes)
artifact = header || raster             (253 bytes)
```

Boundary	Value
Source	`examples/seed_raster.json`
Executable definitions	6
Lowered cells	64 byte-mode `EMIT` cells
`.tmg` bytes	3,200
`.tmg` SHA-256	`2144b904d12420865f77cff4ebccaf4cd41f39603eaaba d691c0c697fe7f482d`
Artifact	`examples/TOM_seed_raster_32x61.pbm`
Artifact bytes	253
Artifact SHA-256	`7025712c6da89ed41f2f0ac3dc62c81a043e4ae2bb3da5 05b0227b8bd255a859`

The raster payload is literally the 244-byte seed. The compiler contains no PBM special case; the nine header bytes and concatenation semantics live in executable definitions.

15.2 Formal operation algebra artifact

`examples/operation_algebra.json` exercises canonical literals, bounded sequence operations, fixed-width arithmetic and bit operations, predicates, binary selection, explicit phase-5 support, phase-6 compatibility, phase-7 guard/event, `State64`, `Cell48`, bounded graph construction, and ordered byte emission.

Its exact 61-byte result is:

```
TOM1TOM{"profile":"A1","version":1}[[0,10],[1,20],[2,30]]\nOK\n
```

Boundary	Value
Executable definitions	49
Lowered cells	17
`.tmg` bytes	944
`.tmg` SHA-256	`4461a4653c7964b188eb6835527d461a5bfda3544406d4 37af45d61d5939f600`
Artifact bytes	61

Boundary	Value
Artifact SHA-256	`320809fa0fcde2f252e691265cf72f9f11de729bfe070e7376799130ff1251f4`

15.3 Migrated polar loop

The primary `examples/polar_loop.json` has no top-level `cells`. Fourteen executable definitions construct the initial state and all eleven cells in the chain:

```
SDF0 -> JIT1 -> KIN2 -> KLEIN -> PHI/HINGE -> LSYS -> CONE -> EMIT
```

The seeded program preserves the legacy final state and complete legacy trace.

Boundary	Value
Lowered cells	11
`.tmg` bytes	656
`.tmg` SHA-256	`7ef5e982f1f230503b86b5d935a86fc5b2de7eb146bdd1712d4ad7864513eb9e`

15.4 Migrated exact-19 rule

The primary `examples/exact19_rule.json` has no top-level `cells`. Six executable definitions construct the initial state, one `CONE` relation cell, two symbolic `EMIT` leaves, and the final program. It emits token 19 for `rho = 19` and token 0 for `rho = 18`.

Boundary	Value
Lowered cells	3
`.tmg` bytes	272
`.tmg` SHA-256	`a8ea712b53c7257ee900bb7c6aa5147b7165dfebd2382ed3e572c4bfce8d4533`

The archived legacy source and binary remain under `examples/legacy/` and reproduce byte-for-byte.

16 Validation and provenance capsule

16.1 Required boundary evidence

A conforming release MUST record:

Boundary	Required evidence
Seed	Exact 244 bytes, no terminal newline, canonical digest.
Registry	Content hash, seed binding, token count, phase order.
Source	Raw file hash and canonical source-document hash.
Definitions	Every content hash, operation, type, dependencies, inputs, phase, ordinal, and output hash.
Graph	Deterministic order and selected root.
Lowering	Complete definition-to-cell crosswalk.
Program	Byte length and SHA-256 of each `.tmg`.
Execution	Python/C equality for final State64, every trace row, every branch, lineage, and every EMIT record.
Artifact	Python/C byte equality, expected length, and expected digest.

Boundary	Required evidence
Replay	Fresh rebuild from the source capsule with identical hashes.
Rejection	Stable rejection of specified invalid sources.

16.2 Recorded validation result

The authoritative emitted record is `validation/genesis_certificate.json`.

The shipped run records:

- 58 Python tests passed;
- 18 validation checks passed and 0 failed;
- full Python/C equality for seed raster, operation algebra, polar loop, exact-19 accept, and exact-19 reject;
- exact Python/C artifact equality for both byte artifacts;
- clean source-capsule replay with identical `.tmg` and artifact hashes;
- fourteen deterministic rejection cases.

16.3 Deterministic rejection set

The reference validator records rejection of:

1. one-byte seed mutation;
2. bad definition content hash;
3. unresolved dependency;
4. dependency cycle;
5. duplicate canonical cell key;
6. invalid successor ID;
7. global cell-budget overflow;
8. ambiguous phase/ordinal order;
9. handwritten top-level cells in seeded mode;
10. codomain mismatch;
11. unknown formal operation;
12. per-definition limit mismatch;
13. non-zero recursive definition depth;
14. unreachable definition.

These are source grammar, identity, type, and finite-resource rules. They do not inject a runtime correction, safe route, confidence gate, damping term, or restoration force.

16.4 TOMAGI-emitted validation certificate

The certificate is not written by a report-specific byte generator. The host validator computes generic boundary values, fills exact `{"$value":"..."}` placeholders in `validation/genesis_certificate.template.json`, rehashes the resulting definitions, compiles them, executes the resulting `.tmg`, and reconstructs canonical JSON bytes from ordered byte-mode `EMIT` records.

Certificate boundary	Value
Executable definitions	6
Source bytes	36,813
Source SHA-256	`b8d314b58257ef49b4c3e75de799e8b3009a9239b68c85d5ad0c3de7d0afc45f`
`.tmg` bytes	244,592
`.tmg` SHA-256	`8fed1bcf679524d7e2e0359b5c9bb3c72480f752d9a1113dc88c21781f424413`
Emitted certificate bytes	20,372

Certificate boundary	Value
Emitted certificate SHA-256	`81e05da7c42328bb749139078baa1d6aa79bf0d39476a11f7891b68e1550daa3`
Compile-manifest SHA-256	`71462caeecc5287e4e7c0791c05d72e2f8c0ba242b40de8db17fc39cabf544bb`
Python/C certificate state	Equal
Python/C certificate trace	Equal
Python/C certificate EMIT sequence	Equal
Python/C certificate bytes	Equal

17 Determinism and replay theorem

17.1 Statement

Let:

- `g` be the exact canonical seed bytes;
- `R` be a hash-valid profile-1.0 token registry bound to `g`;
- `D` be a schema-valid seeded source document whose definitions are hash-valid, acyclic, uniquely scheduled, type-correct, reachable, and within budgets;
- `C` be a conforming profile-1.0 compiler;
- `E` be a conforming TOMAGI ABI 1.0 evaluator;
- `M` be the profile-1.0 byte materializer.

Then repeated evaluation of:

```
M(E(C(g, R, D)))
```

produces the same ordered definition manifest, the same key-sorted `Cell48` table, the same `.tmg` bytes, the same transition trace and lineage, the same ordered `EMIT` sequence, and the same materialized artifact bytes.

17.2 Proof sketch

1. Exact seed verification fixes `g` byte-for-byte.
2. Canonical JSON fixes every content-hash input byte-for-byte.
3. Unique `(phase, ordinal)` slots plus an acyclic dependency graph fix definition order.
4. Every formal operation is finite, deterministic, typed, and budgeted.
5. Unique unsigned 64-bit keys fix cell sorting.
6. Successor IDs are resolved only after that fixed sort.
7. TOMAGI ABI serialization is explicit little-endian fixed-width encoding.
8. TOMAGI transitions are deterministic integer functions of program bytes and input state.
9. `EMIT` chunks are decoded with an exact bit layout and appended in trace order.
10. Therefore every observable boundary is a pure function of the same validated literal inputs.

The theorem does not assert semantic equivalence for non-conforming backends or source documents rejected by the profile.

18 Build and replay commands

From the package root:

```
make validate
```

The equivalent explicit commands include:

```
PYTHONPATH=src/python python -m tomagi compile \
  examples/seed_raster.json examples/seed_raster.tmg \
  --manifest examples/seed_raster.compile_manifest.json
```

```
PYTHONPATH=src/python python -m tomagi materialize \
  examples/seed_raster.tmg examples/TOM_seed_raster_32x61.pbm \
  --trace-output examples/seed_raster.python_trace.json

cc -std=c99 -O2 -Wall -Wextra -Wpedantic -Isrc/c \
  src/c/tomagi.c src/c/tomagi_cli.c -o build/tomagi-c

./build/tomagi-c examples/seed_raster.tmg \
  --trace-json validation/seed_raster.c_trace.json \
  --materialize validation/TOM_seed_raster_32x61.c.pbm

PYTHONPATH=src/python python tools/run_genesis_validation.py
```

19 Evidence scope and deliberate non-claims

This release demonstrates the finite seed-to-bytes causal chain and CPU backend agreement. It does not claim:

- that the canonical string executes without the shipped grammar, registry, definitions, and evaluator;
- that one parity bit is an error-correcting code;
- that `SDF0@Def` makes every stored value numerically zero;
- that topology creates an undeclared physical force or energy;
- universal constant-time execution or compression;
- automatic learning of undeclared semantics;
- new GLSL, WGSL, OpenCL, FPGA, or physical-GPU execution evidence;
- independent proof of priority, patentability, or worldwide novelty.

20 Package map

Key profile files are:

```
TOM_GENESIS_PROPOSAL.md
TOM_seed_genome_2026-09-01.txt
spec/TOM_SEEDED_COMPILATION_PROFILE_1_0.md
spec/tom_seeded_program.schema.json
spec/seeded/token_registry_1_0.json
src/python/tomagi/seed.py
src/python/tomagi/seeded.py
src/python/tomagi/materialize.py
src/python/tomagi/template.py
src/c/tomagi.h
src/c/tomagi.c
src/c/tomagi_cli.c
examples/seed_raster.*
examples/operation_algebra.*
examples/polar_loop.*
examples/exact19_rule.*
examples/legacy/*
validation/genesis_certificate.*
validation/validation_summary.json
checksums/SHA256SUMS.txt
```

The original TOMAGI 1.0 formal report, schemas, catalogs, GPU source mappings, and source crosswalk remain included. The new profile is the executable bridge from the TOM-SRS canonical seed to that unchanged machine.